Datenbanken

Relationale Algebra

Relationales Modell

Domäne	Endliche Menge von Werten desselben Typs ($\approx$ Datentyp)
Attribut	Eigenschaft mit eindeutigem Namen, genau eine Domäne $\text{dom}(A)$
n-Tupel	Menge von n zusammengehörigen Attributwerten
Relationsschema	(Format, Heading, Relationsvariable) Menge von Attributen
Relation	Schema + Menge von Tupeln dazu (Ausprägung $\text{ext}(R)$)

- Relation = endliche **Menge**: keine Duplikate, keine Tupel-Reihenfolge; Attributreihenfolge egal
- **Notation:** Schema $R(A_1, A_2, ..., A_n)$; Tupelmenge $\{\langle a, b, c\rangle, ...\}$ (Format immer angeben)
- **Äquivalenz** $R_1 \sim R_2$: durch Umordnen der Attribute ineinander überführbar (gleiche Domänen/Bedeutung)

Schlüssel

- **Schlüssel:** nicht-leere Attributmenge, identifiziert Tupel eindeutig
- **Schlüsselkandidat:** minimale Schlüsselmenge; jede Relation hat mind. einen (Format selbst)
- **Surrogatschlüssel:** künstlicher Schlüssel, wenn nichts passt
- **Primärschlüssel (PK):** gewählter Kandidat, auf den ein Fremdschlüssel zeigt
- **Fremdschlüssel (FK):** Attributkombination, die auf PK einer anderen Relation zeigt; PK-FK explizit definieren (gleicher Name genügt nicht)

Grundbegriffe

- **RA:** Operatoren + Regeln zum Rechnen mit Relationen; Relationen = Operanden; Ausdrücke = Anfragen (queries)
- **Prädikat:** berechenbarer Ausdruck $\rightarrow$ wahr/falsch
- **Kriterien Abfragesprache:** Ad-hoc-Formulierung, deklarativ („was“, nicht „wo“), optimierbar (wenige Operatoren, Optimierungsregeln), **abgeschlossen** (Ergebnis wieder Relation $\rightarrow$ schachtelbar), mengenorientiert (nicht tuple-at-a-time), sicher (keine Endlosschleife/unendliches Ergebnis)

Selektion σ (entfernt Tupel)

$R' = \sigma_P(R)$, unär

- Prädikat P: logischer Ausdruck aus Attributen/Konstanten $(\wedge, \vee, \neg, =, <, \geq, ...)$, für jedes Tupel geprüft
- Ergebnis: **gleiches Schema**, Tupel mit P wahr; kann leer sein (z. B. $\sigma_{1=0}(r) \rightarrow$ nur Schema, keine Tupel)
- Z. B. $\sigma_{\text{City = 'Zürich'} \wedge \text{Discount} \geq 0.15}(\text{Customers})$
- Kommutativ: $\sigma_{\Phi_1}\big(\sigma_{\Phi_2}(r)\big) = \sigma_{\Phi_2}\big(\sigma_{\Phi_1}(r)\big) = \sigma_{\Phi_1 \wedge \Phi_2}(r)$

Projektion π (entfernt Attribute)

$R' = \pi_{A_1,...,A_k}(R)$, unär

- Schema = Attributliste; Projektion auf nicht vorhandene Attribute $\rightarrow$ **Fehler**
- **Erweitert:** $\pi_{E \rightarrow x,...}(R)$, Ausdruck E aus Attributen, Konstanten, arithm./String-Operatoren, Funktionen; Ergebnis heisst x. Z. B. $\pi_{3 \cdot A \rightarrow X, B+C \rightarrow D, C}(R)$, $\pi_{\text{SQR}(A) \rightarrow X}(R)$
- Duplikate werden entfernt RDBMS/SQL entfernen Duplikate nicht automatisch
- **Nicht** vertauschbar: $\pi_A(\pi_B(r)) \neq \pi_B(\pi_A(r))$ i. A.

Umbenennung ρ

- Gleiche Attributnamen in versch. Relationen qualifizieren: $R.A$, $S.A$
- $\rho_{\text{neu}}(R)$ (auch `R AS neu`): Relation umbenennen, z. B. für Auto-Join
- $\rho_{S(D,E)}(R(A, B))$: $R.A \rightarrow S.D$, $R.B \rightarrow S.E$

Kartesisches Produkt $\times$

$R' = R \times S$, binär

- Jedes Tupel aus R mit jedem aus S kombiniert $\rightarrow$ $|R \times S| = |R| \cdot |S|$
- Schema: R konkateniert mit S; gleichnamige Attribute bleiben beide $\rightarrow$ qualifizieren ($R.B$, $S.B$) oder umbenennen
- In SQL nicht kommutativ: $R \times S \neq S \times R$ (Schema-Reihenfolge; per π überführbar)

Theta-Join $\bowtie_P$ (allg. Verbund)

$R \bowtie_P S = \sigma_P(R \times S)$

- Join-Prädikat P frei: beliebiger log. Ausdruck mit θ-Vergleichen, $\theta \in \{=, <, >, \leq, \geq, \neq\}$
- Schema wie Kreuzprodukt (alle Attribute, ggf. qualifizieren $R.B$, $S.B$)
- Verallgemeinerung des Natural Join, in Praxis Normalfall

Verbundvarianten (-Joins)

Natural	$R \bowtie S$: Gleichheit in allen gemeinsamen Attributen (Name, Domäne, Bedeutung), gemeinsame nur einmal im Schema. Keine gemeinsamen Attribute $\rightarrow$ Kreuzprodukt.
Equi	P nur Gleichheiten, mit $\wedge$ verknüpft. Natural Join $\subset$ Equi-Join (Equi-Join: Attribute dürfen versch. heissen, beide bleiben im Schema)
Auto	$R_1 \bowtie R_1$, braucht Umbenennung ρ
Cross	$R_1 \times R_2$ (kartesisches Produkt)
Semi	$\pi_{r_1..r_n}(R_1 \bowtie R_2)$: Join, projiziert auf Attribute von R_1 (oder R_2)

Mengenoperatoren $\cup$, $\cap$, $\setminus$

Binär; R, S müssen **gleiches Schema** haben. Ergebnis: gleiches Schema, ohne Duplikate.

Op.	Tupel in	kommutativ
$R \cup S$	R ODER S	ja
$R \cap S$	R UND S	ja
$R \setminus S$ $(R - S)$	R ABER NICHT S	nein

Bag-Algebra (Multimengen)

SQL arbeitet mit **Bags**: identische Tupel nicht unterscheidbar, aber Anzahl Vorkommen = **Multiplizität** m.

σ, $\times$, $\bowtie$	Wie RA, Duplikate wie normale Tupel
π	Wie RA, Duplikate **nicht** entfernen
$\cup$ bag union	$\max(m_R, m_S)$
$\sqcup$ bag concatenation	$m_R + m_S$
$\cap$	$\min(m_R, m_S)$
$\setminus$	$\max(m_R - m_S, 0)$
δ Duplikatelimination	Entfernt Duplikate: Bag $\rightarrow$ Relation

Outer Joins $\bowtie$, $\bowtie$, $\bowtie$

Alle Tupel des linken ($\bowtie$ left), rechten ($\bowtie$ right) bzw. beider ($\bowtie$ full) Operanden ins Resultat; ohne Join-Partner $\rightarrow$ fehlende Werte mit **NULL** auffüllen.

- NULL ist **kein Wert**, sondern Indikator für fehlenden Wert; verursacht Probleme $\rightarrow$ möglichst vermeiden

$L(A, B, C)$	$R(C, D, E)$	$L \bowtie R$
a_1, b_1, c_1 a_2, b_2, c_2	c_1, d_1, e_1 c_3, d_2, e_2	a_1, b_1, c_1, d_1, e_1 a_2, b_2, c_2, NULL, NULL NULL, NULL, c_3, d_2, e_2

$\bowtie$: nur Zeilen 1–2; $\bowtie$: Zeilen 1 und 3; $\bowtie$: nur Zeile 1.

Aggregation & Gruppierung

- **Aggregatfunktion:** fasst Attributwerte mehrerer Tupel zusammen: Summe, Durchschnitt, Min, Max, Anzahl
- Über alle Tupel oder pro **Gruppe**: Multimenge von Tupeln mit gleichen Werten im Gruppierungskriterium (Attributkombination), `GROUP BY a` $\rightarrow$ ein Tupel pro Wert von a mit $f(a)$
- Aggregation, Gruppierung, Sortierung: nicht algebraisch

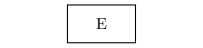
Äquivalenzen

Kommutativität	$\sigma_\Phi(\sigma_\psi(r)) = \sigma_\psi(\sigma_\Phi(r))$ $\pi_A(\sigma_\Phi(r)) = \sigma_\Phi(\pi_A(r))$, falls Φ nur Attr. aus A $r \bowtie s = s \bowtie r$ (Format verschieden)
Assoziativität	$r \bowtie (s \bowtie t) = (r \bowtie s) \bowtie t$
Idempotenz	$\pi_A(\pi_C(r)) = \pi_A(r)$, falls $A \subseteq C$ $\sigma_\Phi(\sigma_\psi(r)) = \sigma_{\Phi \wedge \psi}(r)$
Distributivität	$\pi_A(r \cup s) = \pi_A(r) \cup \pi_A(s)$ $\sigma_\Phi(r \cup s) = \sigma_\Phi(r) \cup \sigma_\Phi(s)$ $\sigma_\Phi(r \bowtie s) = \sigma_\Phi(r) \bowtie s$, falls Φ nur Attr. von r $\pi_{A,B}(r \bowtie s) = \pi_A(r) \bowtie \pi_B(s)$, falls Joinattr. $J \subseteq A \cap B$ $r \bowtie (s \cup t) = (r \bowtie s) \cup (r \bowtie t)$

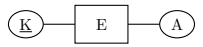
Entity-Relationship (ER)

- Chen 1976; Diagrammsprache für Design von Datenstrukturen
- Dialekt der Vorlesung: liefert Relationen in IDNF (Inclusion Dependency NF), vermeidet NULLs, nur Semantik, die das DBMS überwachen kann

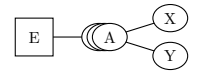
Notation



Entitätstyp: Rechteck, eindeutiger Name. Steht für Menge von Entitäten ($\approx$ Klasse/Objekte) $\rightarrow$ Relation, Entitäten = Zeilen



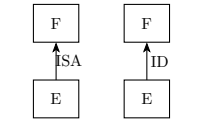
Attribut: Oval, Linie zum Typ; nur einfache (atomare) Werte. Unterstrichen = Primärschlüssel



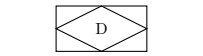
Mehrwertiges (gestapelt) / **zusammengesetztes** Attribut (Unterattribute) $\rightarrow$ in ID-abh. Entitätstyp umwandeln



Beziehungstyp: Rhombus, Pfeile zu Entitätstypen mit Kardinalität 1 / m



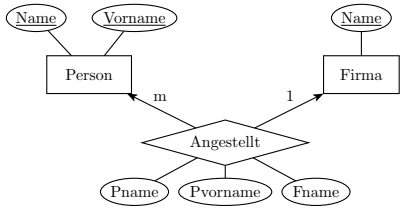
ISA-/ID-abhängiger Entitätstyp: Pfeil vom abhängigen zum übergeordneten Typ



Zusammengesetzter Entitätstyp: Rhombus mit Rechteck umschlossen (Beziehungstyp, an dem Entitätstypen hängen)

- **Pfeil = existentielle Abhängigkeit**; existentiell **unabhängige** Entitätstypen: keine ausgehenden Pfeile, „leben“ für sich
- Primärschlüssel erst wählen, wenn Typ referenziert wird (eingehende Pfeile), sonst nur Schlüssel

Beziehungstyp



- Erbt PK-Attribute der referenzierten Entitätstypen als **Fremdschlüssel** (gleicher Name, ausser bei Namenskonflikt); eigene Attribute möglich (z. B. `seit`)
- Existentiell abhängig von referenzierten Entitätstypen
- Hat **Schlüssel**, keinen PK (ausser er wird referenziert)
- Kardinalität am Pfeil zu E: **1** = höchstens eine Entität aus E pro Kombination der übrigen; **m** = beliebig viele (auch 0)
- Bsp.: pro Person höchstens 1 Firma; Firma beliebig viele Angestellte → Schlüssel {Pname, Pvorname}
- Verwenden, wenn Beziehung **ohne** existentielle Abhängigkeit zw. den Entitätstypen

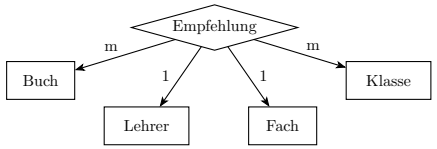
Schlüssel (binär, $E \overset{L_E}{\leftarrow} B \overset{L_F}{\rightarrow} F$)

L_E	L_F	Schlüssel von B
1	1	Schlüssel von E **und** Schlüssel von F (2 Schlüssel)
1	m	Schlüssel von F
m	1	Schlüssel von E
m	m	Schlüssel von E kombiniert mit F

Allgemein
Sei $R(E_1, L_1, ..., E_n, L_n)$ ein Beziehungstyp mit Entitätstypen $E_1, ..., E_n$, Kardinalitäten $L_1, ..., L_n$ und R_K dessen Schlüsselmenge. M_j sei die Menge der FK-Attribute in R zum PK von E_j, wobei $M := M_1 \cup ... \cup M_n$.

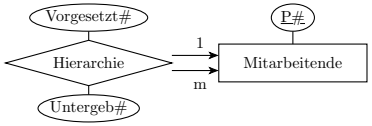
$$R_K = \begin{cases} \{M \setminus M_j | L_j{=}1\} & \exists j{:}L_j{=}1 \\ \{M\} & \forall j{:}L_j{=}m \end{cases}$$

Beziehungen mit $>$ 2 Entitätstypen



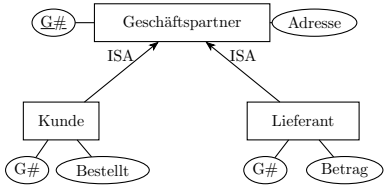
- 1 bei Lehrer: Buch wird einer Klasse in einem Fach von höchstens einem Lehrer empfohlen
- Schlüssel: {B#, F#, K#} und {B#, L#, K#}. **Mehr nicht** herauslesen (z. B. **nicht** „pro Buch höchstens 1 Fach“)
- Semantik knifflig → mehrere binäre Beziehungen erwägen (Semantik dann **nicht** identisch)

Rekursiver Beziehungstyp



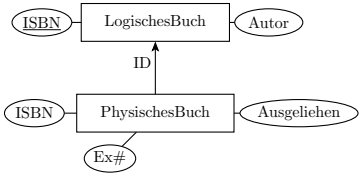
- 1 Entitätstyp genügt; beide FK referenzieren denselben PK P#. Namen so wählen, dass Semantik klar
- Pro Mitarbeitende höchstens 1 Vorgesetzte/r; Vorgesetzte beliebig viele Untergebene

ISA-abhängiger Entitätstyp

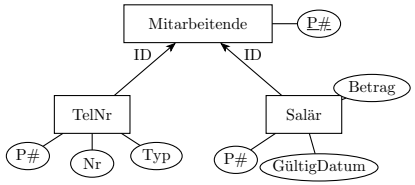


- „is a“: Generalisierung/Spezialisierung; jeder Kunde **ist** Geschäftspartner
- PK von E (Obertyp) ist **Schlüssel** in F (Untertyp): {G#} in Kunde und Lieferant
- Entität kann mehreren Untertypen angehören (Kunde **und** Lieferant)
- Gemeinsame Attribute nur im Obertyp, Schlüssel werden geerbt
- Sinnvoll, wenn Spezialisierungen sich deutlich unterscheiden (Attribute, Beziehungen) **und** Obertyp gemeinsame Struktur auffängt

ID-abhängiger Entitätstyp



- Hierarchie: nur innerhalb des Obertyps identifizierbar (wie Kind per Vorname in Familie)
- Inerhalb eines ID-abhängigen Entitätstyps braucht es ein weiteres Attribut, das die Entität eindeutig identifiziert. (z. B. Ex# für PhysischesBuch)
- **Komplexe Attribute** (mehrwertig/ zusammengesetzt) → ID-abh. Entitätstypen:

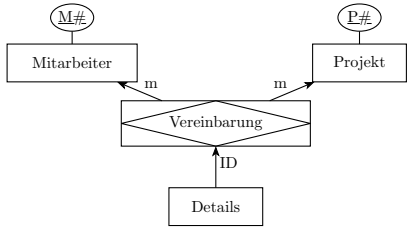


Schlüssel {P#, Nr} bzw. {P#, GültigDatum}; Mitarbeitende ohne Telefon möglich, TelNr nicht ohne Mitarbeitende

ISA vs. ID

ISA	Spezialisierung; PK M des Obertyps ist Schlüssel im Untertyp (1:1)
ID	Hierarchie (1:n); $M \cup N$ Schlüssel, N = mind. 1 differenzierendes Attribut

Zusammengesetzter Entitätstyp



- Wenn an Beziehungstyp Entitätstyp(en) hängen sollen (z. B. beliebig viele Details pro Vereinbarung)
- Beziehungstyp wird umgewandelt (Rechteck darum), erhält PK {M#, P#}
- Details ID-abhängig (mehrere pro Vereinbarung) → Schlüssel z. B. {M#, P#, D#}

Design-Hinweise

- Gleiche Information an **einem** Ort (Beziehungstyp statt mehrwertigem Attribut auf beiden Seiten → sonst Konsistenz nicht gesichert)
- Keine veränderlichen/abgeleiteten Werte speichern: GebDatum statt Alter
- Systemgrenzen ziehen: nur benötigte Daten modellieren
- Produkt $\neq$ Exemplar unterscheiden
- Versch. Aussagen → versch. Beziehungstypen (z. B. „hat im Sortiment“ vs. „hat geliefert“), Attribute wie Preis ggf. in beiden (Lieferpreis $\neq$ Katalogpreis)
- Teilmengen (z. B. Teillieferungen): Kardinalität auf m ändern + Attribut (Menge) in Beziehungstyp

Korrektes ER-Diagramm

Aus leerem Diagramm durch Folge folgender Regeln erzeugbar.

Regel	Voraussetzung ⇒ Ergebnis
1 Unabh. ET	keine ⇒ neues Rechteck (eindeutiger Name)
2 Beziehungstyp	$E_1, ..., E_n$ ($n \geq 1$) Rechtecke oder umschl. Rhomben ⇒ neuer Rhombus mit 1/m-Pfeilen zu den E_j
3 Attribut	F Rechteck, Rhombus oder umschl. Rhombus ⇒ Oval mit Strich zu F, Name eindeutig in F
4 Zusammenges. ET	D Rhombus ⇒ Rechteck um D
5/6 ID-/ISA-abh. ET	F Rechteck oder umschl. Rhombus ⇒ neues Rechteck mit ID/ISA-Pfeil zu F

Schlüssel
- Unabhängiger ET: ≥ 1 Schlüssel; hat er eingehende Pfeile → PK wählen
- Beziehungstyp: FK und Schlüssel gemäss Kardinalitäten
- Zusammengesetzter ET: PK wählen
- E ID-/ISA-abhängig von F: PK in F, FK und Schlüssel in E

Umwandlung in Relationen (kanonische Abbildung)
- Rekursiv dem Aufbau des Diagramms entlang
- Jedes Kästchen (Rechteck, Rhombus) → Relation mit seinen Attributen (Domänen wählen)
- Schlüssel, FK, PK werden übernommen

Normalisierung

Ziel: Redundanzen und Anomalien vermeiden, die bei „drauflos“ definierten Tabellen entstehen.

Probleme

Bsp. `Mitarbeiter(Name, M#, GebDat, Abt#, AName, AMgr)`

Redundanz	Gleiche Info mehrfach gespeichert (AName, AMgr pro MA) → Speicherverschwendung
Insert-Anomalie	Neue Werte müssen konsistent mit bestehenden sein; MA ohne bekannte Abteilung → NULLs; Abteilung ohne MA nicht erfassbar
Delete-Anomalie	Letzten MA einer Abteilung löschen → Abteilungsinfo verloren
Update-Anomalie	Änderung (z. B. AMgr) muss in allen Kopien erfolgen, sonst inkonsistent
Join-Anomalie	Zerlegung über Nicht-Schlüssel (z. B. `(Filiale, Kredit#, Betrag)` + `(Betrag, Kunde)`) → Join liefert zusätzliche, falsche Tupel: **lossy join**

Funktionale Abhängigkeit

Sein U eine nicht-leere Menge von Attributen. Eine funktionale Abhängigkeit hat die Form $X \rightarrow Y$ mit $X, Y \subseteq U$

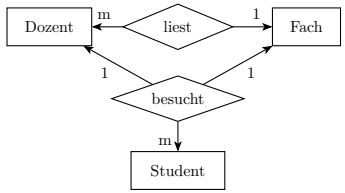
- Bsp. `Mitarbeiter(M#, Abt, Div)`: {Abt, Div} → {Abt}, {M#} → {Abt, Div}, {Div} → {Div}
- Kurz: $AB \rightarrow C$ für $\{A, B\} \rightarrow \{C\}$

Normalformen

1NF	Nur **atomare** Wertebereiche; keine zusammengesetzten/mengenwertigen Attribute → pro Wert eine Zeile (z. B. ein Hobby pro Zeile). Erleichtert Anfragen
2NF	1NF + jedes Nichtschlüsselattribut **voll** funktional abhängig von **jedem**

	Schlüsselkandidaten (vom ganzen, nicht nur von Teil)
3NF	2NF + keine **transitiven** Abhängigkeiten (Attribut hängt über anderes Nichtschlüsselattribut vom PK ab)
BCNF	Alle Attribute hängen von einem Schlüssel, dem ganzen Schlüssel und nichts als dem Schlüssel ab. Verletzt z. B., wenn Teil eines Schlüssels von anderem Attribut abhängt

- **2NF-Bsp.:** `(Verein, Spieler, Stadt, Salär)`, Schlüssel {Verein, Spieler}, aber Verein → Stadt → `(Verein, Stadt)` + `(Verein, Spieler, Salär)`
- **3NF-Bsp.:** `(Spieler_ID, Spieler, Verein, Gründungsjahr)`, Spieler_ID → Verein → Gründungsjahr → `(Spieler_ID, Spieler, Verein)` + `(Verein, Gründungsjahr)`
- **BCNF-Bsp.:** `(Student, Fach, Dozent)`, Schlüssel {Student, Fach}, Dozent → Fach (Dozent liest nur 1 Fach) → nicht BCNF. Zerlegung `(Student, Dozent)` + `(Dozent, Fach)` zerstört {Student, Fach} → Dozent. Korrekte ER-Lösung: Unabhängiges unabhängig modellieren, 2 Abhängigkeiten → 2 Beziehungen:



Normalisierungstheorie vs. Design

- Zwei Wege: 1. Vermeidung durch Designmethodik, 2. Normalisierungstheorie (Hinweise, wie Tabellen zu zerlegen sind)
- Wünschenswert: **BCNF**
- Korrekte ER-Diagramme vermeiden die Probleme von vornherein; mit den Regeln einfach BCNF erreichbar

SQL DDL, DML

Grundlagen

- **Tabellen** statt Relationen: streng genommen **relationale Bags** (Duplikate erlaubt, Abfragen können Duplikate liefern)
- Tupel **nicht geordnet** → kein „gib 10. Tupel“; Attributreihenfolge beliebig, einmal festgelegt aber relevant ($\langle A, B\rangle \neq \langle B, A\rangle$ als Schlüssel)
- Jedes Tupel eindeutig identifizierbar → Unique-Schlüssel nötig
- **ER → Tabellen:** jeder Entitäts- **und** Beziehungstyp → Relationenformat → Tabelle; alle FK-Attribute aufführen, PK unterstreichen; Spalten = Attribute (Struktur), Zeilen = Tupel (Daten)

	DDL (Struktur)	**DML** (Inhalt)
Erzeugen	`CREATE`	`INSERT`
Ändern	`ALTER`	`UPDATE`
Löschen	`DROP`	`DELETE`

DDL

Schema (Datenbank)

```
"CREATE SCHEMA" dbName
    ["AUTHORIZATION" userName] ";"
"DROP SCHEMA" dbName ["CASCADE"] ";"
```

- Auch `CREATE DATABASE`; `dbName` eindeutig; `userName` = Besitzer, nur er darf löschen
- `DROP` löscht per Default nur leeres Schema; `CASCADE` löscht alle Elemente ohne Rückfrage
- **Elemente:** Tabellen, Constraints (DB-, Tabellen-, Attribut-Ebene), Indizes, Stored Procedures, Trigger (letzte 3 nicht standardisiert/proprietär)

Datentypen

`CHAR(n)`, `CHARACTER(n)`	Zeichenkette, fixe Länge
`VARCHAR(n)`, `CHAR VARYING(n)`	Zeichenkette, variable Länge
`INT`, `INTEGER`	Ganzzahl
`REAL`	Fliesskommazahl
`NUMERIC(p,s)`, `DECIMAL(p,s)`	Festkommazahl

Domain

```
"CREATE DOMAIN" domainName "AS"
    dataType ["(" domain ")"]
    [attributeConstraintDef] ";"
"DROP DOMAIN" domainName ";"
```

- Constraint auf **DB-Ebene**: Wertebereich gilt im ganzen Schema, einmal zentral definiert/gepflegt; v. a. für Schlüssel-Attribute, ausführliche Formate
- Bsp.: `CREATE DOMAIN ssn_type AS CHAR(9);` → Verwendung: `ssn ssn_type,`

CREATE TABLE

```
"CREATE TABLE" tableName
  "(" tableElementDef {"," tableElementDef} ");"

tableElementDef = columnDef | tableConstraintDef

columnDef = attributeName
    dataType ["(" domain ")"]
    [attributeConstraintDef]

attributeConstraintDef =
    ["CONSTRAINT" constraintName]
    {"DEFAULT" defaultClause | "NOT NULL"
     | "CHECK" "(" checkCondition ")"}

defaultClause =
    "NULL" | constant | systemVariable

tableConstraintDef =
    ["CONSTRAINT" constraintName]
    tableConstraintType

tableConstraintType = CHECK mit mehreren Attr.
  | "UNIQUE (" attr {"," attr} ")"
  | "PRIMARY KEY (" attr {"," attr} ")"
  | "FOREIGN KEY (" attr {"," attr} ")"
    "REFERENCES" tableName
    ["(" attr {"," attr} ")"]
    [FK-Trigger {FK-Trigger}]
```

```
CREATE TABLE person (
  name varchar(20) NOT NULL,
  vorname varchar(20) DEFAULT ('jim'),
  alter int CHECK (alter > 20),
  accept CHAR(1) CONSTRAINT aval
    CHECK (accept='Y' OR accept='N'),
  CONSTRAINT personPK PRIMARY KEY (name, vorname)
);
```

Constraints

- **Benennen empfohlen:** nur benannte Constraints (einfach) änderbar/löschbar
- **PostgreSQL:** `NOT NULL` nicht benennbar (`CONSTRAINT c NOT NULL` geht nicht; `CHECK(B IS NOT NULL)` theoretisch, unüblich) → `ALTER ... SET/DROP NOT NULL`
- **CHECK:** Ausdruck → true/false; bei Dateneingabe automatisch geprüft; vergleicht mit Konstanten oder Attributen **derselben** Tabelle. Mehrere Attribute → Table-Constraint (nach Attributdefinitionen). Zentrale Einschränkung, sonst muss **jede** Anwendung prüfen
- **UNIQUE:** mehrere pro Tabelle möglich (Schlüssel); DBMS prüft Eindeutigkeit `CONSTRAINT BesitztUni UNIQUE(pNr, Modell)`
- **PRIMARY KEY:** höchstens 1 pro Tabelle; nötig, wenn per FK referenziert, sonst genügt UNIQUE `CONSTRAINT BestPosPK PRIMARY KEY(bestNr, posNr)`
- **FOREIGN KEY:** minimal `FOREIGN KEY (a) REFERENCES T`; ohne Attributliste → **PK** von `T` referenziert (andere Attribute möglich, abzuraten). DBMS sichert **referentielle Integrität** (Eingabe, Löschen)

FK-Trigger

```
FK-Trigger = ("ON DELETE" | "ON UPDATE")
  ("NO ACTION" | "SET NULL" | "SET DEFAULT"
   | "CASCADE")
```

Automatische Aktionen, ausgelöst durch Änderung **in der referenzierten Tabelle**:

`ON UPDATE`	Tupel in referenzierter Tabelle geändert/eingefügt
`ON DELETE`	Tupel in referenzierter Tabelle gelöscht
`NO ACTION`	Aktion verboten, falls FK danach auf keinen gültigen PK mehr zeigt
`SET NULL/DEFAULT`	FK auf NULL/Default setzen (SET NULL vermeiden)
`CASCADE`	Update: FK-Werte angepasst. Delete: referenzierende Tupel gelöscht.

```
CONSTRAINT PasstZuFK1 FOREIGN KEY (Modell)
  REFERENCES Smartphone
  ON UPDATE CASCADE ON DELETE CASCADE,
```

→ Modell umbenannt: `PasstZu` übernimmt neuen Namen; Modell gelöscht: zugehörige `PasstZu`-Tupel gelöscht

ALTER TABLE

```
"ALTER TABLE" tableName action ";"
action = "ADD" column | "DROP" column
  | "ALTER" column | "ADD" constraint
  | "DROP" constraint
```

- Jede Ebene änderbar: Table → TableElement → ElementConstraintDef
- `ADD`: neues Attribut, Table-Constraint, Attribut-Constraint (teilweise)
- `DROP` Attribut/Constraint mit `RESTRICT` (verboten, solange Constraints/Views es referenzieren) oder `CASCADE` (abhängige Unique/FK werden „gebrochen“)

```
ALTER TABLE HatGeliefert ADD menge INT NOT NULL
  CONSTRAINT hatGelMengePos CHECK(menge>0);
ALTER TABLE HatGeliefert
  DROP CONSTRAINT hatGelMengePos;
ALTER TABLE HatGeliefert ALTER menge SET DEFAULT 1;
ALTER TABLE HatGeliefert DROP eanNr CASCADE;
ALTER TABLE Zubehoerteil ALTER name SET NOT NULL;
ALTER TABLE Zubehoerteil ALTER name DROP NOT NULL;
```

- `SET/DROP NOT NULL`: proprietäre PostgreSQL-Erweiterung
- Mehrere Änderungen in einem `ALTER` = Erweiterung: PostgreSQL `ADD a int, ADD b char(20)`, Oracle `ADD (a int, b char(20))`

DROP TABLE

```
"DROP TABLE" tableName {"," tableName}
    ["CASCADE" | "RESTRICT"] ";"
```

- Löscht Tabelle inkl. Struktur; hängen Objekte ab (z. B. FKs) → `CASCADE` nötig

DML

INSERT

```
"INSERT INTO" tableName ["(" attrList ")"]
    ("VALUES (" valueList ")") | query ";"
```

- Fügt immer **ganze Tupel** ein; Reihenfolge `attrList` ↔ `valueList` muss übereinstimmen

```
INSERT INTO Salaer VALUES (12, 35000, 24, 5, 2003);
INSERT INTO Salaer (pNr, betrag)
  VALUES (12, 35000);
```

UPDATE

```
"UPDATE" tableName "SET" attr "=" wert
    {"," attr "=" wert}
    ["WHERE" searchCondition] ";"
```

- 4 Grundrechenoperationen erlaubt: `UPDATE Salaer SET betrag = 1.05 * betrag;`
- Ohne `WHERE` → alle Tupel

```
UPDATE Salaer SET betrag = 1.05 * betrag WHERE pNr
= 12;
```

DELETE

```
"DELETE FROM" tableName
    ["WHERE" searchCondition] ";"
```

- Löscht immer **ganze Tupel**; **ohne Bedingung → ganze Tabelle geleert!**
- Einzelnes Attribut „löschen“ → `UPDATE ... SET betrag = NULL WHERE ...`

```
DELETE FROM Salaer WHERE pNr = 12;
```

SQL Queries

Query

```
Query = subquery {("UNION" | "INTERSECT"
  | "EXCEPT") ["ALL" | "DISTINCT"] subquery} ";"
```

- `INTERSECT` = ∩, `EXCEPT` = \; Default hier **`DISTINCT`**

- Beliebig verschachtelbar, liefert immer **eine** Tabelle

SELECT (Projektion)

```
subquery = "SELECT" ["ALL" | "DISTINCT"]
    attributeList "FROM" Datenquelle
attributeList = columnSpec {"," columnSpec} | "*"
columnSpec = scalarExpr ["AS" neuerName]
scalarExpr = (columnRef | literal)
    {("+" | "-" | "*" | "/") scalarExpr}
columnRef = [[schema "."] table "."] attr
```

- Default **`ALL`**; `SELECT a` $= \pi_a$ (Bag), `SELECT DISTINCT a` $= \delta(\pi_a)$
- `*` = alle Attribute der Datenquelle; `AS` = Rename
- `columnRef` so weit qualifizieren, bis eindeutig
- Literal: `SELECT 1 FROM T` → je Tupel von `T` eine `1`
- **`DISTINCT` sparsam**: internes Sortieren $\approx n \log n$; überflüssig, wenn Resultat Schlüssel enthält

```
SELECT DISTINCT betrag FROM Salaer;
SELECT betrag * 1.025 AS neu FROM Salaer;
```

Datenquelle (FROM)

```
Datenquelle = tableExpr | joinExpr
tableExpr = [schema "."] table [["AS"] alias]
joinExpr = (tableExpr | "(" joinExpr ")")
  ["NATURAL" | ("LEFT" | "RIGHT" | "FULL") "OUTER"
   | "CROSS"] "JOIN" tableExpr ["ON" joinCondition]
joinCondition = attrComparison
    {("AND" | "OR") attrComparison}
attrComparison = ([table "."] attr | literal)
  theta ([table "."] attr | literal)
theta = "<" | "<=" | "=" | ">=" | ">" | "<>"
```

- `ON` nötig ausser bei `NATURAL` und `CROSS`
- `CROSS JOIN` = $\times$; `NATURAL JOIN` = $\bowtie$ (gleichnamige Attribute)
- Attributnamen nur innerhalb Tabelle eindeutig → bei Equi-Join `T.attr` qualifizieren
- Equi-Join hat **mehr Spalten** als Natural Join (dieser behält gemeinsame Attr. nur einmal); Natural Join immer als Equi-Join formulierbar
- Zusammengesetzter Schlüssel → **alle** Attribute in Join-Condition

```
SELECT name, ort FROM Person JOIN Mitarbeiter
  ON Person.pNr = Mitarbeiter.pNr;
SELECT BestellPosition.bestNr, posNr,
  BestellPosition.menge AS BestellMenge,
lieferDatum
FROM (BestellPosition JOIN Enthaelt
    ON BestellPosition.bestNr = bNr AND posNr =
pNr)
  JOIN Lieferung
    ON bNr = Lieferung.bestNr AND lNr = liefNr;
```

- **Subquery als Datenquelle**: PostgreSQL verlangt Alias (Bereichsvariable)

```
SELECT Name FROM (SELECT * FROM Person
  WHERE Name = 'Müller') AS x;
```

WHERE (Selektion)

```
searchCondition = "WHERE" ["NOT"] logicalComparison
  {("AND" | "OR") ["NOT"] logicalComparison}
logicalComparison = (attr | literal)
  {compareOp (attr | literal)}
compareOp = "<" | "<=" | "=" | ">=" | ">" | "<>"
  | "LIKE"
```

- Tupelfilter: nur Tupel mit Ausdruck **true** ins Resultat; `WHERE P` $= \sigma_P$
- `SELECT a FROM R WHERE P` $= \pi_a(\sigma_P(R))$

`LIKE 'M__er'`	Nur Strings; `_` = genau 1 Zeichen, `%` = beliebig viele
`BETWEEN x AND y`	Bereich (inkl. Grenzen)
`IS NULL`	**Nicht** `= NULL`!
`[NOT] IN (...)`	Liste von Literalen oder Subquery

```
SELECT name FROM Zubehoerteile NATURAL JOIN PasstZu
WHERE modell IN ('Galaxy', 'iPhone');
```

Kurzform Join

`FROM a, b WHERE a.x = b.x` ≡ `FROM a JOIN b ON a.x = b.x`

```
SELECT g.bname, g.rname, s.bsorte
FROM gast AS g, sortiment AS s
WHERE g.rname = s.rname AND g.bname = 'Anderegg';
```

Ordnung

- Resultat-Reihenfolge nicht garantiert
- `ORDER BY a, b [DESC]` (Default aufsteigend)
- Gleichwertige Tupel können weiterhin umsortiert werden → Sortierkriterium muss Schlüssel enthalten für stabile Ordnung
- Spalten immer in definierter Reihenfolge

Lexikographische Ordnung

Nicht direkt ausdrückbar:
$\langle a, b, c\rangle < \langle x, y, z\rangle \Leftrightarrow (a < x) \vee (a = x \wedge b < y) \vee (a = x \wedge b = y \wedge c < z)$

```
-- zwischen <Meier, Hans> und <Schmid, Joseph>
WHERE (Name = 'Meier' AND Vorname >= 'Hans')
   OR (Name > 'Meier' AND Name < 'Schmid')
   OR (Name = 'Schmid' AND Vorname <= 'Joseph')
```

EXISTS

- Existenzbedingung: true, falls Subquery ≥ 1 Tupel liefert; „innere Schleife“ pro äusserem Tupel (korreliert)
- Gleiche Tabelle zweimal → **Bereichsvariablen** (Aliase) nötig, sonst `Vorname = Vorname` immer wahr

```
-- Besucher, deren Vorname ein anderer auch hat
SELECT * FROM Besucher AS x
WHERE EXISTS (SELECT 1 FROM Besucher AS y
  WHERE x.Vorname = y.Vorname
    AND NOT (x.Name = y.Name));
```

By Justin Iven Müller (2026)
github.com/JustinIven/zhaw-cheatsheets